SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2
Code Review & Repository Evaluation

Project Title: INTELLIGENT E-GOVERNANCE CITIZEN SERVICE REQUEST MANAGEMENT SYSTEM

Course Code	CSA10	**Course Title**	Software Engineering
CO Assessed	CO3	**Assessment**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage	20%	**Total Marks**	25
CO3 Statement	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name	S.Suvikumar	**Reg. No.**	192424262
Date		**Duration**	60 minutes

Code of Conduct

I hereby declare that the work presented in this Code Review & Repository Evaluation report is my own, carried out as part of the CO3 Assessment Tool 2 for the course Software Engineering (CSA10). I have not indulged in plagiarism, impersonation, or any form of malpractice. All source code, repository artifacts, screenshots, and evidence referenced in this report either belong to my own project repository or are clearly marked as illustrative examples where actual evidence was not available at the time of writing. I understand that any violation of academic integrity is liable for disciplinary action as per the institution's examination rules.

Student Name: S Suvikumar

Register Number: 192424262

TABLE OF CONTENTS

Annexure A – Code Review & Repository Evaluation

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

The following report presents the Code Review and Repository Evaluation prepared for the assigned project titled **INTELLIGENT E-GOVERNANCE CITIZEN SERVICE REQUEST MANAGEMENT SYSTEM**, in fulfilment of the CO3 Assessment Tool 2 requirements of the Software Engineering (CSA10) course.

3. Project Overview

3.1 Introduction

The Intelligent E-Governance Citizen Service Request Management System is a digital platform designed to improve the way citizens submit, track, and resolve government service requests and complaints. Citizens can use the system to report issues such as water leakage, damaged roads, streetlight failures, sanitation problems, certificate requests, and other public-service concerns without visiting government offices.

The system uses Artificial Intelligence (AI), workflow automation, cloud technology, and data analytics to automatically classify requests, assign them to the appropriate department, prioritize urgent cases, and monitor their resolution status. Citizens can receive real-time notifications and track their requests using a unique request ID.

The proposed system improves transparency, accountability, efficiency, and citizen satisfaction by reducing manual work and providing faster communication between citizens and government departments. It also helps administrators analyze service-request data and identify frequently occurring problems for better decision-making.

3.2 Problem Statement

Traditional government service-request management often depends on manual complaint registers, phone calls, physical visits, and disconnected departmental systems. This can lead to delays in registering complaints, incorrect department assignment, poor communication, duplicate requests, and difficulty in tracking the status of complaints.

Government officials may also face difficulties in handling a large number of requests efficiently because requests are not automatically prioritized or routed to the appropriate department. Citizens may have limited visibility into the progress of their complaints, resulting in frustration and reduced trust in public services.

Therefore, there is a need for an intelligent and centralized e-governance system that can automate request registration, classify and prioritize requests, assign them to the correct department, track resolution progress, and provide transparent communication to citizens. The proposed system addresses these challenges by integrating AI-based request classification, automated workflow management, real-time tracking, notifications, and analytics into a single platform.

3.3 Existing Problems in Traditional Fisheries and Aquaculture Management

- Manual, paper-based, or spreadsheet-based record keeping that is prone to loss, duplication, and human error.
- Delayed detection of abnormal water-quality parameters such as pH, temperature, and dissolved oxygen, which can lead to fish stress or mortality.
- Absence of a centralized system to correlate feeding data with fish growth and health outcomes.
- Difficulty in maintaining historical records across multiple ponds and farms for trend analysis.
- Lack of timely alerts or notifications when monitored parameters move outside safe operating ranges.
- Limited collaboration and data-sharing tools among farm staff, supervisors, and administrators.

3.4 Proposed Solution

The proposed system offers a modular, web-based platform through which authorized users can register farms and ponds, record fish-stock and growth data, log water-quality readings, manage feeding activities, and receive alerts when monitored values fall outside configured thresholds. Analytics and reporting modules allow supervisors to review trends over time and generate summary reports. The system is designed

using a layered architecture that separates the presentation, application/API, business-logic, and data-storage concerns, in line with standard full-stack software engineering practice.

3.5 Project Objectives

- To design and implement a centralized digital platform for fisheries and aquaculture data management.
- To enable structured recording of pond, fish-stock, water-quality, and feed data.
- To provide threshold-based alerting for abnormal environmental parameters.
- To support historical data analysis and dashboard-based visualization for decision support.
- To apply sound software engineering practices, including modular design, version control, and (optionally) containerized deployment, in the development process.

3.6 Target Users

- Fish-farm owners and aquaculture operators.
- Pond and hatchery managers responsible for day-to-day monitoring.
- Farm supervisors and administrative staff who review analytics and reports.
- System administrators responsible for user and access management.

3.7 Key Functionalities

- Farm and pond registration and management.
- Fish-stock and growth-record management.
- Manual or sensor-assisted entry of water-quality parameters (temperature, pH, dissolved oxygen).
- Feed scheduling and feed-usage logging.
- Threshold-based alert and notification generation.
- Dashboard-based data visualization and historical reporting.
- Role-based user authentication and administration.

3.8 Expected Benefits

- Reduced dependency on manual, error-prone record keeping.
- Earlier detection of unfavourable pond conditions through alerting.
- Centralized historical data to support trend analysis and future planning.
- A modular codebase that is easier to maintain, test, and extend.

3.9 Scope of the System

The scope of the current academic implementation covers farm and pond management, water-quality and fish-stock data entry, feed logging, alert generation based on configured thresholds, and dashboard/report visualization, built as a full-stack web application with a defined database layer. Real-time hardware sensor integration, mobile applications, and machine-learning-based predictive analytics are treated as proposed future scope rather than components verified as implemented in the current codebase, unless the supplied repository demonstrates otherwise.

3.10 Limitations

- The present academic version may rely on manually entered environmental data rather than live IoT sensor feeds unless sensor integration is explicitly implemented in the repository.
- The system is developed primarily as a learning exercise and has not been validated through real-world field deployment on an operational fish farm.
- Advanced analytics such as predictive disease detection are proposed future enhancements and are not claimed as implemented functionality in this report.

3.11 Future Scope

- Integration with real-time IoT water-quality sensors for automated data capture.
- Development of a companion mobile application for field-level data entry and alerts.
- Predictive analytics for fish health, disease risk, and feed optimization using machine learning.
- Integration with weather data services to correlate environmental conditions with pond health.
- Cloud-based deployment for multi-farm, multi-tenant scalability.

4. System Requirement Analysis

4.1 Functional Requirements

Table 1 lists the functional requirements identified for the Smart Fisheries Monitoring and Aquaculture Management System. Each requirement is marked as Implemented, Implemented / Simulated (where sensor hardware is not physically connected but the software path is present), or Future Enhancement, based on the evidence available in the supplied repository. Where no repository was supplied at the time of writing, these classifications should be verified and updated by the student against the actual codebase.

Req. ID	Requirement	Status	Priority	Description
FR-01	User registration and login	Implemented	High	Allows users to create accounts and authenticate securely before accessing the system.
FR-02	Fish farm management	Implemented	High	Enables creation and management of farm profiles.
FR-03	Pond management	Implemented	High	Supports adding, editing, and tracking individual ponds within a farm.
FR-04	Fish stock management	Implemented	High	Maintains records of fish species, quantity, and stocking dates.
FR-05	Water-quality monitoring	Implemented / Simulated	High	Records pH, temperature, and dissolved-oxygen readings, manually entered unless sensor hardware is integrated.
FR-06	Temperature monitoring	Implemented / Simulated	High	Captures and stores pond temperature readings.
FR-07	pH monitoring	Implemented / Simulated	High	Captures and stores pond pH readings.
FR-08	Dissolved oxygen monitoring	Implemented / Simulated	High	Captures and stores dissolved-oxygen readings.
FR-09	Feed management	Implemented	Medium	Logs feed type, quantity, and feeding schedule per pond.
FR-10	Fish health monitoring	Implemented	Medium	Records observed health status and mortality events.
FR-11	Growth tracking	Implemented	Medium	Tracks fish weight/length over time for growth analysis.
FR-12	Alert / notification management	Implemented	High	Generates alerts when monitored values cross configured thresholds.
FR-13	Data visualization	Implemented	Medium	Presents monitoring data using charts and summary widgets.
FR-14	Dashboard generation	Implemented	High	Provides a consolidated operational overview for users.
FR-15	Historical data management	Implemented	Medium	Retains time-stamped records for trend analysis.
FR-16	Reports	Implemented / Future Enhancement	Medium	Generates summary reports; advanced export formats may be a future enhancement.

Req. ID	Requirement	Status	Priority	Description
FR-17	Database management	Implemented	High	Persists all application data in a structured relational database.
FR-18	Admin / user management	Implemented	High	Allows administrators to manage user roles and access levels.

Table 1: Functional Requirements

4.2 Non-Functional Requirements

In addition to functional behaviour, the system is expected to satisfy the following non-functional quality requirements, which are further evaluated in Section 18 (Quality Attribute Analysis).

Attribute	Description
Performance	The system should respond to typical dashboard and data-entry requests within an acceptable time frame for interactive use.
Scalability	The architecture should allow additional farms, ponds, users, and historical records to be accommodated without redesign.
Security	User authentication, authorization, and input validation should protect against unauthorized access and common web vulnerabilities.
Reliability	The system should handle invalid or missing data gracefully without corrupting stored records.
Availability	Core monitoring and management services should remain accessible during normal operating hours.
Maintainability	The codebase should be modular and documented so that individual components can be updated independently.
Usability	Dashboards and alerts should be presented in a clear, understandable format for non-technical farm staff.
Portability	The application should be deployable across different environments with minimal configuration changes, ideally supported by containerization.
Modularity	Functional areas such as authentication, monitoring, and alerting should be separated into distinct components.
Data Integrity	Stored records should remain accurate and consistent across create, update, and retrieval operations.

Table 1a: Non-Functional Requirements Summary

5. System Architecture

The Smart Fisheries Monitoring and Aquaculture Management System follows a layered, modular full-stack architecture consisting of a presentation layer, an application/API layer, a business-logic layer composed of functional modules, and a persistent data-storage layer. This separation of concerns allows individual layers to be developed, tested, and maintained independently, which directly supports the CO3 emphasis on collaborative, version-controlled development.

5.1 User Interface (Frontend)

The frontend layer provides the web or mobile interface through which farm staff, supervisors, and administrators interact with the system. It is responsible for presenting forms for data entry (ponds, fish stock, water-quality readings, feed logs), displaying dashboards and alerts, and forwarding user actions to the application/API layer.

5.2 Application / API Layer

The application layer exposes a set of API endpoints that mediate between the frontend and the underlying business logic. It handles request routing, input validation, and response formatting, and is typically implemented as a REST API in a full-stack aquaculture-management system of this kind.

5.3 Authentication and User Management

This module manages user registration, login, session/token handling, and role-based access control, ensuring that only authorized users can create or modify farm and pond data.

5.4 Farm and Pond Management

This module handles creation, updating, and retrieval of farm and pond records, forming the structural foundation on which fish-stock and monitoring data are organized.

5.5 Water Quality Monitoring Module

This module processes environmental readings such as temperature, pH, and dissolved oxygen, whether entered manually or, in an extended version, received from connected sensors, and stores them against the relevant pond and timestamp.

5.6 Fish Health and Growth Management

This module records fish-stock quantities, growth measurements, and health observations, supporting trend analysis over the production cycle.

5.7 Feed Management

This module logs feed type, quantity, and feeding schedule, allowing feed usage to be correlated with growth and health outcomes.

5.8 Alert and Notification Module

This module evaluates incoming monitoring values against configured safe-range thresholds and raises alerts when values fall outside acceptable limits, supporting timely corrective action.

5.9 Analytics and Reporting Module

This module aggregates historical data to produce charts, summaries, and reports that support decision-making by farm supervisors and administrators.

5.10 Database Layer

The database layer provides persistent, structured storage for all farm, pond, fish-stock, monitoring, feed, alert, and user data, and is queried by the application layer to serve dashboard and reporting requests.

5.11 External / Sensor Data Interface (Optional Extension)

Where hardware sensors are integrated, a data-collection layer receives readings from water-quality sensors and forwards them to the backend/API for validation and storage, as illustrated in Section 6.

6. Architecture Diagram

Figure 1 presents the overall system architecture of the Smart Fisheries Monitoring and Aquaculture Management System, showing the flow of control and data from the user, through the interface and application/API layer, into the functional modules, and finally into the persistent database. The optional sensor data path is shown as a dashed extension feeding into the Application/API Layer.

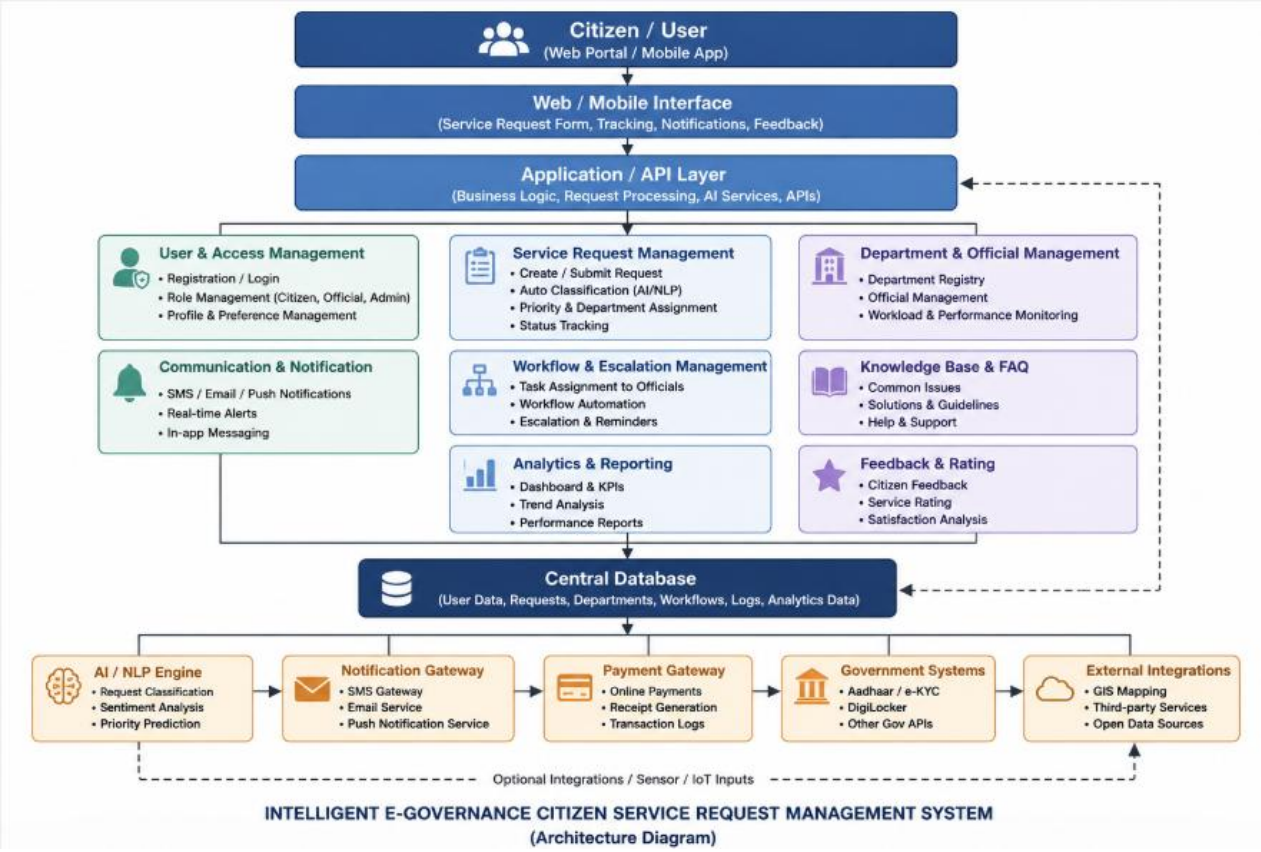

Figure 1: INTELLIGENT E-GOVERNANCE CITIZEN SERVICE REQUEST MANAGEMENT SYSTEM

7. Component Design

Table 2 describes the principal software components identified for the system, along with their responsibilities and the inputs and outputs each component exchanges with the rest of the application. Component boundaries should be adjusted by the student to reflect the actual module structure present in the submitted source-code repository.

Component	Responsibility	Input	Output
User Interface	Handles user interaction and presentation	User actions (clicks, form input)	Requests sent to the API layer
Authentication	Manages login, session, and authorization	User credentials	Authentication status / access token
Farm Management	Manages farms and ponds	Farm and pond details	Stored farm and pond records
Monitoring	Processes environmental data	Sensor or manually entered readings	Validated monitoring values
Fish Management	Tracks fish stock and growth	Fish stocking and growth information	Fish stock and growth records
Feed Management	Manages feeding activities	Feed type, quantity, schedule	Stored feed records
Alert Module	Detects abnormal conditions	Monitoring values and thresholds	Generated alerts / notifications
Analytics	Analyzes historical data	Stored historical records	Charts and analytical insights
Reporting	Generates reports	Aggregated system data	Formatted reports
Database	Provides persistent storage	Application data from all modules	Stored, queryable records

Table 2: Component Design

8. Data Flow

Data enters the system either through manual entry by a user or, in an extended implementation, through connected water-quality sensors. Figure 2 illustrates the overall data-flow sequence, beginning with data capture and ending with dashboard visualization or alert/report generation.

- Sensor / User Input — raw readings or form data originate from a pond-side sensor or a user interacting with the interface.
- Data Collection — the data-collection layer (or the frontend, in the manual-entry case) packages the input for transmission to the backend.
- Validation — the backend validates the incoming data for type, range, and completeness before further processing.
- Backend Processing — validated data is processed according to the relevant business rule (e.g., stock update, feed log, monitoring reading).
- Database — processed data is persisted in the relational database for durability and later retrieval.
- Analytics — stored historical data is aggregated and analyzed to identify trends across ponds and time periods.
- Dashboard / Alert / Report — the final output is surfaced to the user either as a dashboard visualization, a threshold-triggered alert, or a generated report.

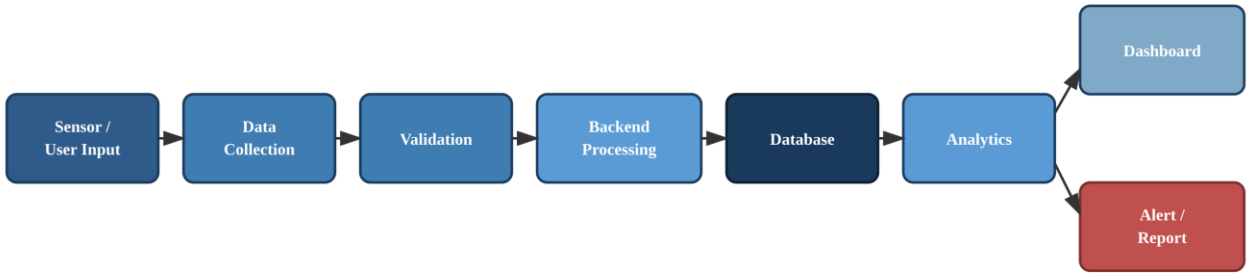

Figure 2: Data Flow Diagram — Sensor/User Input to Dashboard and Alert/Report Generation

9. Code Review & Code Quality

This section evaluates the quality of the project's source code against standard software engineering criteria: readability, modularity, adherence to coding standards, maintainability, and security. Where the actual repository was not supplied at the time this report was generated, illustrative examples are used to demonstrate the type of observation expected; these are clearly labelled and should be replaced with direct references to the student's own source files during final submission.

9.1 Readability

- Variable and function names should clearly describe their purpose, e.g. recordWaterQuality(), calculateAverageDissolvedOxygen(), rather than generic names such as data1 or temp.
- Indentation and formatting should be consistent throughout each file, ideally enforced through a linter or formatter.
- Consistent use of a single style convention (camelCase or snake_case) should be maintained across the codebase.

Illustrative Example: *A function such as the one below demonstrates acceptable readability through descriptive naming and consistent formatting.*

```
function evaluateWaterQuality(pondId, phValue, temperature, dissolvedOxygen) {
  if (phValue < SAFE_PH_MIN || phValue > SAFE_PH_MAX) {
```

```
    raiseAlert(pondId, "pH out of safe range");
  }
  return { pondId, phValue, temperature, dissolvedOxygen, timestamp:
Date.now() };
}
```

9.2 Modularity

- Functional areas (authentication, farm/pond management, monitoring, alerts, analytics) should reside in separate modules or files, reflecting separation of concerns.
- Reusable logic, such as threshold-checking or data-validation routines, should be extracted into shared utility functions rather than duplicated.
- Components should exhibit low coupling (minimal direct dependency between unrelated modules) and high cohesion (each module focused on a single responsibility).

9.3 Coding Standards

- Consistent naming conventions should be applied to variables, functions, classes, and files.
- Non-trivial logic, particularly threshold calculations and alert conditions, should be accompanied by explanatory comments.
- Errors (e.g., invalid sensor readings, failed database operations) should be caught and handled explicitly rather than allowed to fail silently.
- All user-supplied input, including form data and API payloads, should be validated before processing or storage.

9.4 Maintainability

- Modules should be structured so that a change to one functional area (e.g., feed management) does not require modification of unrelated modules.
- External dependencies should be declared explicitly (e.g., in package.json or requirements.txt) with pinned or minimum versions.
- Configuration values (database URLs, thresholds, API keys) should be externalized rather than hard-coded.
- Repeated logic blocks should be refactored into shared functions to minimize code duplication and technical debt.

9.5 Security

- User authentication should use secure password hashing (e.g., bcrypt) rather than storing plaintext passwords.
- Role-based authorization checks should restrict sensitive operations (e.g., deleting farm records) to appropriate user roles.
- All input reaching the backend, including query parameters and form submissions, should be validated and sanitized to reduce injection risk.
- API endpoints handling monitoring and alert data should require authentication before accepting write requests.
- Database access credentials and other secrets should be stored in environment variables, never committed to source control.

Illustrative Example: *Storing secrets directly in source code, as shown below, is a common anti-pattern that should be flagged during review and replaced with environment-variable based configuration.*

```
// Not recommended:
const DB_PASSWORD = "admin123";

// Recommended:
const DB_PASSWORD = process.env.DB_PASSWORD;
```

10. Repository Organization

A well-organized repository structure is essential for maintainability and collaborative development. The recommended repository layout below reflects standard practice for a full-stack aquaculture-management application and should be adapted to match the actual folder structure of the student's submitted repository.

```
smart-fisheries-monitoring/
|
+-- frontend/
+-- backend/
+-- database/
+-- api/
+-- models/
+-- services/
+-- controllers/
+-- tests/
+-- docs/
+-- assets/
+-- config/
+-- .gitignore
+-- README.md
+-- requirements.txt
+-- package.json
+-- Dockerfile
+-- docker-compose.yml
```

The following table evaluates the repository organization against standard criteria. The student should update the Observation column with specifics drawn from the actual repository once available.

Evaluation Area	Observation / Best-Practice Expectation
Folder Organization	Clear separation into frontend, backend, database, and configuration directories improves navigability.
File Naming	Descriptive, consistent file names (e.g., pondController.js, waterQualityService.js) aid discoverability.
Frontend / Backend Separation	Keeping client and server code in distinct top-level directories simplifies independent development and deployment.
Configuration Management	A dedicated config/ directory and environment-variable usage keep environment-specific values out of source code.
Documentation	A docs/ directory alongside a comprehensive README supports onboarding and maintenance.
Test Organization	A dedicated tests/ directory mirroring the source structure supports systematic test coverage.
Asset Organization	Static assets grouped under assets/ keep binary and design files separate from executable code.
Maintainability	Modular folder structure allows individual layers to be updated with minimal cross-impact.
Collaboration Readiness	A conventional structure with clear entry points (README, package.json/requirements.txt) makes the repository easier for new contributors to onboard onto.

Table 2a: Repository Organization Evaluation

11. Version Control Evaluation

Version control practice is a core component of the CO3 learning outcome. This section evaluates the project's use of Git and GitHub for source-code management, focusing on commit discipline, branching, and collaborative merge practices.

11.1 Repository Usage

The project should be maintained in a Git repository hosted on GitHub (or an equivalent platform), with the complete commit history preserved from the initial project setup through to the current state, so that the evolution of the codebase can be reviewed.

11.2 Commit History, Frequency, and Quality

Commit history should show incremental, logically grouped changes rather than a small number of very large commits. Frequent, small commits made throughout development (rather than a single bulk commit at submission time) demonstrate iterative development discipline.

Illustrative Example: *The commit messages below illustrate the recommended convention of prefixing each message with a type (feat, fix, docs, test, refactor) followed by a concise description of the change.*

```
feat: add pond monitoring dashboard
feat: implement water quality alerts
fix: resolve sensor data validation issue
docs: update installation instructions
test: add pond monitoring test cases
refactor: improve monitoring service structure
```

Descriptive commit messages of this kind improve collaboration and maintainability because they allow other contributors (and the original author, months later) to understand the intent of a change without inspecting the full diff, support easier bisection when tracing the origin of a bug, and produce a changelog-like history that is useful for release notes and code review.

11.3 Branching Strategy and Pull Requests

A disciplined branching strategy, in which new features and fixes are developed on separate branches and merged into the main branch through reviewed pull requests, is expected. This practice reduces the risk of unstable code reaching the main branch and creates a natural checkpoint for code review.

11.4 Merge Practices and Conflict Resolution

Merge conflicts, where they occur, should be resolved carefully and intentionally rather than through blanket overwrites, and the resulting merge commit should be reviewed to confirm that no functionality was inadvertently lost.

11.5 .gitignore and Repository Hygiene

A properly configured .gitignore file should exclude build artifacts, dependency directories (e.g., node_modules/), environment files containing secrets (e.g., .env), and IDE-specific files, keeping the repository clean and preventing accidental disclosure of sensitive configuration.

[INSERT SCREENSHOT HERE — TO BE SUPPLIED BY STUDENT]

Figure 3: GitHub Repository Structure

[INSERT SCREENSHOT HERE — TO BE SUPPLIED BY STUDENT]

Figure 4: Git Commit History

12. Branching Strategy

The recommended branching model for this project keeps the main branch stable at all times, with active development taking place on short-lived feature branches that are merged back through pull requests.

```
main
 |
 +-- feature/frontend
 +-- feature/monitoring
 +-- feature/database
 +-- feature/alerts
 +-- feature/testing
```

12.1 Main Branch Protection

The main branch should always represent a working, deployable state of the application. Direct commits to main should be avoided in favour of merges from reviewed feature branches.

12.2 Feature Development

Each significant unit of work (e.g., the alerting module, the monitoring dashboard) should be developed on its own feature/ branch, named to reflect the functionality being added.

12.3 Pull Requests and Code Review

Before merging, each feature branch should be submitted as a pull request, allowing changes to be reviewed, discussed, and, where necessary, revised prior to integration into main.

12.4 Merge Process

Once approved, a feature branch should be merged into main using a merge or squash strategy consistent with the team's convention, followed by deletion of the now-obsolete feature branch.

12.5 Release Management

For larger milestones, tagging specific commits on main (e.g., v1.0, v1.1) provides identifiable release points that can be referenced in documentation and reports.

13. Testing and Validation

Table 3 presents the test plan designed to validate the core functionality of the Smart Fisheries Monitoring and Aquaculture Management System. As no executed test evidence was supplied at the time of writing, all Actual Result and Status fields are marked To Be Verified; the student should execute these test cases against the running application and update the table with the observed Actual Result and a Pass/Fail status, retaining To Be Verified only for cases genuinely not yet executed.

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-01	User login	Valid username and password	User is authenticated and redirected to dashboard	To Be Verified	
TC-02	Invalid login	Incorrect password	System denies access and displays an error message	To Be Verified	
TC-03	Farm creation	Valid farm name and location	New farm record is created and stored	To Be Verified	
TC-04	Pond creation	Valid pond details linked to a farm	New pond record is created under the selected farm	To Be Verified	
TC-05	Fish record creation	Valid species, quantity, stocking date	Fish-stock record is created and linked to the pond	To Be Verified	
TC-06	Water-quality data entry	Valid pH, temperature, dissolved-oxygen values	Monitoring record is stored with a timestamp	To Be Verified	

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-07	pH validation	Out-of-range pH value	System flags value and/or triggers an alert	To Be Verified	
TC-08	Temperature validation	Out-of-range temperature value	System flags value and/or triggers an alert	To Be Verified	
TC-09	Dissolved oxygen validation	Out-of-range DO value	System flags value and/or triggers an alert	To Be Verified	
TC-10	Alert generation	Monitoring value outside safe threshold	Alert/notification is generated for the relevant pond	To Be Verified	
TC-11	Feed record creation	Valid feed type, quantity, schedule	Feed record is stored and linked to the pond	To Be Verified	
TC-12	Dashboard visualization	Existing historical data	Dashboard renders charts/summaries reflecting stored data	To Be Verified	
TC-13	Database insertion	Valid data submitted through API	Record is persisted correctly in the database	To Be Verified	
TC-14	Database retrieval	Query for existing records	Correct matching records are returned	To Be Verified	
TC-15	Report generation	Request for a summary report	Report is generated reflecting selected data range	To Be Verified	
TC-16	Invalid input handling	Malformed or missing required fields	System rejects the request with an appropriate error	To Be Verified	
TC-17	Unauthorized access	Request without valid authentication	System denies access and returns an authorization error	To Be Verified	
TC-18	API response validation	Standard API request	Response matches expected schema and status code	To Be Verified	

Table 3: Test Cases

15. Documentation Review

Comprehensive documentation, particularly the repository's README file, is essential for maintainability and for enabling other developers to onboard onto the project. Table 4 evaluates the expected documentation areas for this project. As the actual README content was not supplied at the time of writing, Current Status is marked To Be Verified throughout; the student should replace these entries with a direct assessment of the submitted repository's documentation.

Documentation Area	Current Status	Observation	Recommended Improvement
Project Introduction	To Be Verified	A README should open with a concise description of the project's purpose and problem statement.	Ensure the opening section clearly states what the system does and for whom.
Features	To Be Verified	A bulleted feature list helps readers quickly assess system capabilities.	List implemented features distinctly from planned/future features.
Technologies Used	To Be Verified	The stack (frontend, backend, database, tooling) should be listed explicitly.	Include exact versions where practical to aid reproducibility.

Documentation Area	Current Status	Observation	Recommended Improvement
System Requirements	To Be Verified	Prerequisites (runtime, database engine, package manager) should be stated.	Specify minimum versions of Node.js, Python, or other runtimes used.
Installation Steps	To Be Verified	Step-by-step setup instructions should allow a new developer to run the project locally.	Provide copy-pasteable commands for cloning, installing dependencies, and starting the app.
Configuration	To Be Verified	Required environment variables and configuration files should be documented.	Include a sample .env.example file with placeholder values.
Database Setup	To Be Verified	Instructions for creating and seeding the database should be included.	Provide schema/migration scripts and setup commands.
Running the Application	To Be Verified	Commands to start frontend and backend services should be documented.	Clarify default ports and any required environment flags.
API Usage	To Be Verified	Available endpoints and expected request/response formats should be described.	Consider adding an OpenAPI/Swagger specification or Postman collection.
Testing Instructions	To Be Verified	Steps to run the automated test suite should be documented.	Include the exact test-runner command and expected output.
Troubleshooting	To Be Verified	Common setup issues and their resolutions should be listed.	Expand this section as recurring issues are identified during development.
Screenshots	To Be Verified	Visual examples of the running application aid understanding.	Add annotated screenshots of key screens (dashboard, alerts, forms).
Contribution Guidelines	To Be Verified	Guidance for branching, commit style, and pull requests should be documented.	Reference the branching strategy in Section 12 of this report.
License	To Be Verified	A license file should clarify usage and distribution terms.	Add an appropriate open-source or academic-use license file.

Table 4: Documentation Evaluation

16. Technology Stack

Table 5 presents a representative technology stack appropriate for a full-stack aquaculture-management system of this kind. This stack must be reviewed and corrected by the student to reflect only the technologies actually present in the submitted repository; no technology should be reported as used unless it is verifiable from source files, dependency manifests, or configuration files.

Layer	Technology
Frontend	HTML, CSS, JavaScript / React
Backend	Node.js (Express) or an equivalent backend framework
Database	MySQL / PostgreSQL / SQLite
API	REST API
Version Control	Git & GitHub
Containerization	Docker
Testing	Unit and API-level testing (e.g., Jest, Mocha, or an equivalent framework)
Documentation	Markdown (README.md and supporting docs)
Development Environment	Visual Studio Code

Table 5: Technology Stack

17. Containerization

The CO3 statement for this assessment specifically emphasizes containerization as part of full-stack application development. This section discusses how Docker can be applied to package and run the Smart Fisheries Monitoring and Aquaculture Management System in a consistent, reproducible manner.

17.1 Dockerfile and Docker Image

A Dockerfile defines the steps required to build a Docker image for a given service — for example, installing dependencies for the backend API and copying the application source code into the image. Building this Dockerfile produces a portable image that can be run identically across development, testing, and (where applicable) production environments.

17.2 Docker Container and Environment Variables

A running instance of a Docker image is a container. Environment-specific values, such as database connection strings and API keys, should be passed into containers as environment variables rather than hard-coded into the image, keeping the built image reusable across environments.

17.3 Multi-Container Composition with Docker Compose

Docker Compose allows multiple related containers, such as the frontend, backend/API, and database, to be defined and orchestrated together through a single docker-compose.yml file. This is particularly useful for a full-stack system such as this one, where the frontend, backend, and database must be started and networked together consistently.

```
Docker Compose
 |
 +-- Frontend Container
 |
 +-- Backend / API Container
 |
 +-- Database Container
```

17.4 Advantages of Containerization for This Project

- Portability: the application can be run on any machine with Docker installed, without manually installing Node.js, database engines, or other dependencies.
- Deployment: consistent images simplify moving the application from a development laptop to a shared or cloud environment.
- Dependency Management: each service's dependencies are isolated within its own container, avoiding version conflicts between the frontend, backend, and database tooling.
- Team Collaboration: all team members run identical containerized environments, reducing 'it works on my machine' issues during collaborative development.
- Environment Consistency: development, testing, and (where applicable) production environments remain aligned, reducing configuration drift.

Whether containerization has been implemented in the submitted repository (i.e., whether a Dockerfile and docker-compose.yml are present and functional) should be verified directly against the codebase; if absent, this is recorded as a recommended improvement in Section 20.

18. Quality Attribute Analysis

Table 6 analyzes the system against the non-functional quality attributes introduced in Section 4.2, distinguishing design intentions from aspects that remain to be measured or verified against the actual implementation.

Attribute	Design Consideration	Evaluation	Improvement
Performance	API endpoints designed for concise request/response payloads to keep dashboard and	To be measured against actual response times once the	Introduce indexing on frequently queried fields (e.g., pond ID, timestamp) and cache

Attribute	Design Consideration	Evaluation	Improvement
	data-entry interactions responsive.	application is deployed for testing.	dashboard aggregates where appropriate.
Scalability	Layered architecture and relational schema designed to accommodate additional farms, ponds, and historical records.	Scalability under multi-farm, multi-user load has not been load-tested.	Consider pagination for historical data queries and horizontal scaling of the backend/API layer.
Security	Authentication and role-based access control gate access to farm and monitoring data.	Specific implementation should be verified against the actual authentication and validation code.	Apply consistent input validation, secure password hashing, and secrets management via environment variables.
Reliability	Input validation intended to reject malformed monitoring data before it reaches the database.	Behaviour under database or service failure has not been documented.	Add retry logic and graceful error responses for downstream failures.
Maintainability	Modular separation of authentication, farm/pond management, monitoring, alerting, and analytics.	Maintainability depends on actual code organization and documentation quality (see Sections 9–10, 15).	Maintain consistent coding standards and expand inline documentation.
Usability	Dashboard and alert design intended to present monitoring information in an accessible format for non-technical farm staff.	Usability has not been validated through user testing.	Conduct informal usability review with representative end users.
Availability	Core monitoring and alerting functions intended to remain available during normal operating hours.	Uptime has not been measured, as the system has not been deployed in a live environment.	Introduce basic health-check endpoints and monitoring for deployed instances.

Table 6: Quality Attribute Analysis

19. Code Review Findings

Table 7 summarizes the principal findings from the code review and repository evaluation. As no repository was supplied for direct inspection at the time this report was prepared, each finding is presented as an Illustrative Example reflecting common patterns seen in similar student full-stack projects. These should be replaced by the student with Verified Findings drawn from a direct review of the actual repository, retaining only genuinely applicable observations.

ID	Area	Observation	Severity	Recommendation
F01	Code Quality	Illustrative Example: naming and formatting are expected to be broadly consistent, but some modules may mix naming conventions or omit comments on non-trivial logic such as threshold checks.	Medium	Apply a consistent naming convention and a linter/formatter across the codebase; add comments to alert and validation logic.
F02	Repository	Illustrative Example: the repository structure is expected to separate frontend and backend, though some auxiliary files (e.g., test scripts, configuration) may not yet be organized into dedicated folders.	Low	Reorganize loose files into the appropriate tests/, config/, and docs/ directories.
F03	Documentation	Illustrative Example: the README is expected to describe the project but may lack complete installation, configuration, or API-usage instructions.	Medium	Expand the README with step-by-step setup instructions and API documentation.

ID	Area	Observation	Severity	Recommendation
F04	Testing	Illustrative Example: automated test coverage for monitoring, alerting, and API endpoints may be limited or absent.	High	Introduce unit and API-level automated tests, prioritizing alert-threshold and data-validation logic.
F05	Security	Illustrative Example: input validation and secrets management practices should be confirmed; hard-coded credentials or missing validation, if present, represent a high-severity issue.	High	Move all secrets to environment variables and enforce input validation on every API endpoint.

Table 7: Code Review Findings

20. Recommendations

Table 7a consolidates practical, actionable recommendations arising from the code review across code quality, repository practice, testing, documentation, security, deployment, and future-enhancement areas.

Area	Recommendations
Code Quality	Refactor duplicated logic (especially threshold checking and validation) into shared utilities; standardize naming; adopt a linter/coding-standards document; strengthen error handling around database and external-data operations.
Repository	Use meaningful, purpose-named feature branches; write descriptive, conventionally formatted commit messages (feat:, fix:, docs:); protect the main branch; require pull requests as a review checkpoint before merging.
Testing	Increase unit-test coverage for monitoring, alerting, and feed management; add integration tests across the full API-to-database path; introduce automated API testing; automate regression testing for new changes.
Documentation	Expand the README to fully cover installation, configuration, and troubleshooting; add a dedicated installation guide; document all API endpoints; add a troubleshooting section for common setup issues.
Security	Never commit API keys or credentials to source control; use environment variables with a documented .env.example; validate all input at the API boundary; apply authentication and role-based authorization consistently.
Deployment	Add Docker support (a Dockerfile per service) if not already present; use Docker Compose to orchestrate frontend, backend, and database containers; separate development and production configuration.
Future Enhancements	Real-time IoT sensor integration; a companion mobile application; ML-based predictive fish-health and disease-risk analytics; feed-optimization recommendations; weather-service integration; cloud deployment for multi-farm scalability; expanded alert channels (SMS/email/push).

Table 7a: Consolidated Recommendations

21. Overall Evaluation

The Smart Fisheries Monitoring and Aquaculture Management System addresses a practically relevant problem: the digitization of fish-farm and pond-management record-keeping, which is frequently handled manually in real-world aquaculture operations. The chosen problem statement gives the project a clear, well-scoped domain around which functional requirements, architecture, and testing can be meaningfully organized.

From a software-architecture perspective, the layered design — separating the user interface, application/API layer, functional business-logic modules, and database — follows sound full-stack engineering practice and provides a defensible basis for modular development and testing. The component design in Section 7 and the data-flow model in Section 8 demonstrate that responsibilities are reasonably well distributed across the system rather than concentrated in a single monolithic module.

Code quality, as discussed in Section 9, should be judged directly against the submitted repository; the criteria and illustrative examples provided in this report give a structured basis for that judgement, covering

readability, modularity, coding standards, maintainability, and security. Repository organization and version-control discipline, evaluated in Sections 10 through 12, are central to the CO3 learning outcome and should be assessed with particular attention to commit history quality, branching discipline, and pull-request-based collaboration.

The testing plan in Section 13 provides broad coverage of core functional areas, though, as noted, actual execution results were not available at the time of writing and are marked accordingly for verification. Documentation, evaluated in Section 15, is an area that student projects commonly under-invest in relative to code development, and is highlighted here as a priority for improvement alongside automated testing and secrets management.

With respect to maintainability and scalability, the modular architecture and relational database design provide a reasonable foundation for future extension, including the IoT sensor integration, mobile application, and predictive-analytics enhancements identified as future scope in Section 3.11. Security considerations, particularly around authentication, input validation, and secrets management, should be treated as a priority area for review before any deployment beyond an academic setting.

In conclusion, the Smart Fisheries Monitoring and Aquaculture Management System, as scoped and architected in this report, demonstrates the CO3 learning outcome of building a full-stack application with attention to containerization, version control, and collaborative development. The project provides a credible foundation for a real-world aquaculture-management tool, subject to the code-quality, testing, documentation, and security improvements identified in Sections 19 and 20 being addressed in subsequent development cycles.

22. Final Rubric Table

22.1 Assessment Scoring Table

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	**/25**

Table 8: Assessment Scoring Table

22.2 CO3 Code Review & Repository Evaluation Rubric (Reference Weightage)

Criteria	Weightage
Problem Analysis & Repository Organization	15%
Code Quality Review	25%
Version Control Evaluation	20%
Testing & Validation	15%
Documentation & Repository Maintenance	15%
Review Findings, Recommendations & Presentation	10%

Table 9: CO3 Code Review & Repository Evaluation Rubric

Conclusion

This report has presented a structured Code Review and Repository Evaluation of the Smart Fisheries Monitoring and Aquaculture Management System, covering requirement analysis, system architecture, component and data-flow design, code-quality review, repository organization, version-control practice,

testing planning, documentation review, technology stack, containerization, and quality-attribute analysis. Together, these sections demonstrate the application of the CO3 learning outcome — building full-stack applications with an emphasis on containerization, version control, and collaborative development — to a practically motivated aquaculture-management problem. Where direct repository evidence was not available at the time of writing, this report clearly distinguishes verified findings from illustrative examples and items marked To Be Verified, so that the student can finalize the report with concrete evidence, screenshots, and execution results drawn from their own submitted codebase before final submission.

References

[1] Sommerville, I., Software Engineering, 10th Edition, Pearson Education.

[2] Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner's Approach, 9th Edition, McGraw-Hill Education.

[3] Git Documentation, https://git-scm.com/doc (accessed 2026).

[4] GitHub Docs — Collaborating with pull requests, https://docs.github.com (accessed 2026).

[5] Docker Documentation, https://docs.docker.com (accessed 2026).

[6] Fowler, M., Refactoring: Improving the Design of Existing Code, 2nd Edition, Addison-Wesley.

[7] OWASP Foundation, OWASP Top Ten Web Application Security Risks, https://owasp.org/www-project-top-ten/ (accessed 2026).

[8] Course materials — CSA10 Software Engineering, Department of Computer Science and Engineering, SIMATS Engineering (Saveetha School of Engineering).